Bi-Weekly Research Progress Report

Project: IsoShard Proxy: A Distributed CP-Mode Transaction Coordinator & SQL Optimizer on AWS
Proponent: Sarthak Mushrif
Reporting Period: Weeks 7 & 8
Hours Logged: 20 Hours

1. Executive Summary of Progress

During the fourth reporting period, development focused heavily on the system's networking layer and physical database connectivity. To bypass the latency overhead of standard HTTP/REST protocols, a Custom Binary Codec was developed within the Java Netty pipeline to handle byte-level serialization and deserialization. Furthermore, a highly optimized HikariCP connection pool was established to maintain persistent TCP links between the proxy layer and the simulated AWS RDS shards. Finally, foundational logic for Phase 1 (The "Prepare" Voting Phase) of the Two-Phase Commit (2PC) protocol was drafted.

2. Detailed Task Breakdown (20 Hours)

- **Custom Binary Codec Implementation (8 Hours):** Designed and implemented the `ByteToMessageDecoder` and `MessageToByteEncoder` for the Netty server. The protocol dictates a strict packet structure: a 4-byte payload length header, a 1-byte command identifier, and the UTF-8 encoded SQL payload. This avoids Java string allocation overhead until the packet is fully received.
- **HikariCP Connection Pool Integration (6 Hours):** Configured the Hikari Connection Pool to manage outgoing connections to the physical database shards. This involved setting aggressive connection timeouts, max pool sizing per virtual node, and keep-alive intervals to mimic production AWS environment constraints.
- **Two-Phase Commit - Phase 1 Drafting (6 Hours):** Initialized the `TransactionCoordinator` class. Developed the logic to broadcast the `PREPARE` command to all shards participating in a distributed write operation. Utilized `CompletableFuture` to ensure the Netty EventLoop thread remains unblocked while waiting for the database shards to acknowledge the row locks.

3. Challenges & Roadblocks

- **TCP Packet Fragmentation:** During local load testing, incoming TCP byte streams were occasionally fragmented, causing the Netty decoder to fail when attempting to read the full SQL string. This was resolved by implementing a `LengthFieldBasedFrameDecoder` at the front of the Netty pipeline, which buffers incoming bytes until the complete packet (dictated by the 4-byte header) is assembled.
- **Asynchronous Thread Deadlocks:** Early iterations of the 2PC broadcast logic caused thread starvation when waiting for JDBC connections from the Hikari pool. Refactored the architecture to delegate blocking JDBC calls to a separate, dedicated worker thread pool, keeping the primary Netty I/O threads free.

4. Objectives for Next Reporting Period (Weeks 9 & 10)

- Complete Phase 2 of the Two-Phase Commit protocol (The "Commit/Rollback" Decision Phase).

- Integrate AWS DynamoDB (or a local simulated equivalent) to serve as the high-speed Write-Ahead Log (WAL) for transaction state persistence.
- Conduct end-to-end integration testing of a distributed `UPDATE` transaction across two separate database shards.

Appendix A: Technical Artifact - Custom Binary Decoder

The following Java class demonstrates the zero-copy Netty decoder designed to read the custom binary protocol, resolving TCP fragmentation issues before passing the payload to the SQL parser.

```java
import io.netty.buffer.ByteBuf;
import io.netty.channel.ChannelHandlerContext;
import io.netty.handler.codec.ByteToMessageDecoder;
import java.nio.charset.StandardCharsets;
import java.util.List;

public class IsoShardPacketDecoder extends ByteToMessageDecoder {

    @Override
    protected void decode(ChannelHandlerContext ctx, ByteBuf in, List<Object>
    out) {
        // Wait until we have at least the 4-byte header (payload length)
        if (in.readableBytes() < 4) {
            return;
        }

        in.markReaderIndex();
        int payloadLength = in.readInt();

        // Wait until the entire payload has arrived (handles TCP fragmentation
    )
        if (in.readableBytes() < payloadLength) {
            in.resetReaderIndex();
            return;
        }

        // Read the 1-byte command type (e.g., 0x01 for QUERY, 0x02 for PREPARE
    )
        byte commandType = in.readByte();

        // Read the actual SQL string payload
        CharSequence sqlQuery = in.readCharSequence(payloadLength - 1,
    StandardCharsets.UTF_8);

        // Pass the decoded query object down the pipeline to the AST Compiler
        out.add(new IsoShardRequest(commandType, sqlQuery.toString()));
    }
}
```

Listing 1: IsoShardPacketDecoder.java